Data Structures — Final Exam: Detailed Solutions and Last-Minute Review Guide

This exam paper is worth 100 points in total. The question types cover six categories — multiple-choice, fill-in-the-blank, true-or-false, short-answer, comprehensive, and programming. As a whole the paper emphasizes basic concepts, the core properties of data structures, time-complexity analysis, and classic algorithm principles, while the programming question focuses on basic linked-list implementation. The difficulty is on the basic side and the exam points are highly concentrated, which makes it perfect for grabbing the key points in a last-minute final-exam review.

I. Question-by-Question Detailed Solutions

I. Multiple-Choice Questions (2 points each, 20 points total)

1. Answer: c

Solution: this question tests the search operation on arrays. Arrays support random access (accessing an element by index takes O(1)), but "search for an element" in the question means searching by value; for an unordered array you must traverse from the start, so the time complexity is O(n). For a sorted array, binary search reaches O(log n); the question does not say the array is sorted, so by default it tests sequential search on an unordered array.

Key exam point: the search complexity of the sequential list

2. Answer: b (c is an equivalent wording; the standard wording is last-in, first-out)

Solution: a stack is a linear list restricted so that insertion and deletion happen only at the tail of the list (the top of the stack); its core property is last-in, first-out (LIFO), which can also be stated as "first-in, last-out" — the two wordings are semantically identical. Option a is the property of a queue, and d is the property of an array.

Key exam point: the basic properties of a stack

3. Answer: c (with the typography corrected: 2^{n-1})

Solution: this question tests the properties of binary trees. Any binary tree has at most 2^{k-1} nodes on level k, and the complete binary tree, being a special case of the binary tree, satisfies the same property. Option c of the question contains a typographical error; the correct form is 2^{n-1}.

Key exam point: properties of binary trees; characteristics of complete binary trees

4. Answer: c

Solution: this question tests linked-list insertion. The complete insertion consists of two steps: ① traverse to find the predecessor of the insertion position, with average / worst-case time complexity O(n); ② modify the pointers to complete the insertion, with time complexity O(1). The question does not state that "the predecessor position is known", so by default it tests the time complexity of the complete operation — hence O(n).

Key exam point: insertion in a singly linked list; time-complexity analysis

5. Answer: d

Solution: this question tests the time complexity of sorting algorithms. Bubble sort, insertion sort, and selection sort all have average and worst-case time complexity $O(n^2)$; quicksort's average time complexity is O(n log n), the best of the four options.

Key exam point: comparing the time complexity of basic sorting algorithms

6. Answer: a

Solution: a hash table maps keys directly to storage addresses through its hash function; in the ideal conflict-free scenario a search needs only one computation, and the average search time complexity is O(1) — the most core advantage of the hash table.

Key exam point: the search properties of hash tables

7. Answer: b

Solution: this question tests graph storage structures. The adjacency matrix stores edge information in a two-dimensional array with space complexity $O(n^2)$, suited to dense graphs; the adjacency list stores each vertex's adjacent vertices in linked lists with space complexity O(n+e). The fewer the edges (the sparser the graph), the more obvious the space advantage, so it suits sparse graphs.

Key exam point: the characteristics and applicable scenarios of graph storage structures

8. Answer: c

Solution: the search efficiency of a binary search tree depends on the height of the tree. When elements are inserted in sorted order, the binary search tree degenerates into a singly linked list; the height of the tree is then n, and the worst-case search time complexity is O(n); on average it is O(log n).

Key exam point: the search complexity of binary search trees

9. Answer: c

Solution: heapsort consists of two steps, "building the initial heap + adjusting the heap repeatedly". Among them, building the max-heap takes O(n) time (provable by mathematical derivation — it is not O(n log n)), a high-frequency, easily-missed exam point; the total time complexity of the entire heapsort is O(n log n).

Key exam point: time-complexity analysis of heapsort

10. Answer: d

Solution: the core of divide and conquer is "divide and rule": break the big problem into several smaller subproblems of the same type, solve them recursively, and merge the results. Quicksort is the classic application of divide and conquer: a pivot splits the array into two parts, each sorted recursively. Binary search belongs to "decrease and conquer" (only one subproblem is kept each time) and is usually not treated as a typical representative of divide and conquer.

Key exam point: typical divide-and-conquer algorithms

II. Fill-in-the-Blank Questions (2 points each, 20 points total)

1. Answer: O(n)

Solution: same reasoning as insertion — deleting a node requires first traversing to find the predecessor of the node to be deleted, a step of time complexity O(n); the pointer modification itself is O(1). By default, the complexity of the complete deletion operation is tested.

2. Answer: mapping keys to storage addresses (hash addresses) in the hash table

Solution: the hash function is the core of a hash table; its role is to convert an element's key value into the index of the corresponding storage location, achieving a direct "key → address" mapping.

3. Answer: stack

Solution: the core of depth-first search (DFS) is "go all the way to the end, backtrack when stuck"; its backtracking nature matches the stack's "last-in, first-out" perfectly. A recursive DFS is essentially using the system stack; a non-recursive implementation maintains a stack structure manually.

4. Answer: queue

Solution: the core of breadth-first search (BFS) is "spreading outward level by level", processing vertices in the order they were visited, which matches the queue's "first-in, first-out" property.

5. Answer: O(n log n)

Solution: on average, each partition of quicksort splits the array evenly into two parts; the recursion depth is log n and the total work per level is O(n), so the average time complexity is O(n log n).

6. Answer: 1

Solution: the definition of the AVL tree (balanced binary search tree) is: the absolute value of the height difference between the left and right subtrees of any node in the tree does not exceed 1.

7. Answer: Kruskal

Solution: there are two classic minimum-spanning-tree algorithms for graphs: Prim's algorithm and Kruskal's algorithm.

8. Answer: directed acyclic (DAG)

Solution: topological sorting orders the vertices of a directed acyclic graph (DAG) so that for any directed edge u→v, vertex u comes before v. A graph with cycles cannot be topologically sorted.

9. Answer: queue

Solution: level-order traversal of a binary tree (visiting level by level, top to bottom and left to right) needs to process nodes in visit order; it is implemented with the help of a queue's first-in, first-out property.

10. Answer: prefix (optimal prefix / lossless compression)

Solution: Huffman coding is an optimal prefix code based on a Huffman tree: no character's code is a prefix of another character's code, which guarantees unambiguous decoding while also achieving lossless data compression.

III. True-or-False Questions (2 points each, 10 points total)

1. Answer: ×

Solution: a stack is a last-in, first-out (LIFO) data structure; first-in, first-out (FIFO) is the property of a queue.

2. Answer: √

Solution: a red-black tree maintains the tree's balance through color constraints and rotation operations; it is a common self-balancing binary search tree whose search, insertion, and deletion all take O(log n).

3. Answer: ×

Solution: a linked list does not support random access; searching for an element requires traversal starting from the head node, with time complexity O(n). O(log n) is the complexity of binary search on a sorted array.

4. Answer: √

Solution: the common methods for resolving hash collisions fall into two classes: open addressing (linear probing, quadratic probing, etc.) and separate chaining.

5. Answer: ×

Solution: Dijkstra's algorithm solves the single-source shortest-path problem on weighted graphs (non-negative weights); shortest paths in unweighted graphs are usually found with breadth-first search (BFS).

IV. Short-Answer Questions (5 points each, 20 points total)

1. The main differences between stacks and queues and their typical application scenarios

Answer points:

- Core differences: ① Different operation restrictions: a stack allows insertion and deletion only at the top of the stack (the tail of the list); a queue allows insertion only at the rear and deletion only at the front. ② Different properties: a stack is last-in, first-out (LIFO); a queue is first-in, first-out (FIFO).
- Typical applications: Stack: the function-call stack, expression evaluation, bracket matching, converting recursion into iteration, depth-first search (DFS). Queue: breadth-first search (BFS), level-order traversal of binary trees, task scheduling, data buffers.

2. The inorder traversal process of a binary search tree and its properties

Answer points:

- Traversal process: recursively traverse each node in the order "left subtree → root node → right subtree"; that is, first traverse all nodes of the left subtree, then visit the current root node, and finally traverse all nodes of the right subtree.
- Core property: the inorder traversal of a binary search tree yields a sequence sorted in ascending order. This is the most important property of the BST and a common way to check whether a tree is a binary search tree.

3. The definition of hash collision and common resolutions

Answer points:

- Definition: when two different keys key1 and key2 are computed by the hash function to the same hash address (i.e. f(key1)=f(key2)), the phenomenon is called a hash collision.
- Common resolutions: ① Open addressing: on collision, keep looking for the next empty storage slot in the hash table, including linear probing, quadratic probing, and so on. ② Separate chaining: store all elements with the same hash address in the same singly linked list; each slot of the hash table stores the head pointer of the corresponding list.

4. The working principles of DFS and BFS

Answer points:

- Depth-first search (DFS): Starting from the source vertex, after visiting the current vertex, recursively visit its unvisited adjacent vertices, going deep along one path; when the current vertex has no unvisited neighbors, backtrack to the previous vertex and continue visiting its other neighbors, until all vertices are visited. Usually implemented with a stack (recursion / an explicit stack).
- Breadth-first search (BFS): Starting from the source vertex, first visit the source vertex, then visit all of its direct neighbors in turn, then spread outward level by level to the neighbors' neighbors, until all vertices are visited. Usually implemented with a queue.

V. Comprehensive Questions (10 points each, 20 points total)

1. Designing a sorting algorithm for an unordered array and analyzing its complexity

Reference solution (using quicksort as the example):

- Algorithm design: ① Partition: pick a pivot element from the array and split the array into two parts — the elements on the left are all less than or equal to the pivot, the elements on the right are all greater than or equal to the pivot — and the pivot lands in its final correct position. ② Recurse: run quicksort recursively on the left and right subarrays. ③ Termination condition: when a subarray has length 0 or 1, it is already ordered and the recursion terminates.
- Time complexity: Average case: each partition splits the array evenly into two parts; the recursion depth is O(log n) and the total work per level is O(n), giving an average time complexity of O(n log n). Worst case: when the array is already sorted, each partition splits the array into parts of length 0 and n-1; the recursion depth is O(n), and the total time complexity is $O(n^2)$.
- Space complexity: Quicksort is a recursive algorithm that needs stack space to store the recursion-call information. On average the recursion depth is O(log n), so the space complexity is O(log n); in the worst case the recursion depth is O(n), so the space complexity is O(n).

2. The basic steps of Dijkstra's algorithm and a worked example

- Basic steps: Dijkstra's algorithm solves the single-source shortest-path problem on weighted graphs (non-negative weights). The steps are as follows: ① Initialization: set the shortest distance of the source s to 0 and the shortest distances of all other vertices to infinity; maintain the set S of vertices whose shortest distances are settled, initially empty. ② Select a vertex: among the vertices not yet added to S, pick the vertex u with the smallest current shortest distance and add it to S. ③ Relaxation: for every neighbor v of the vertex u, check whether the path "source → u → v" is shorter than v's current shortest distance; if so, update v's shortest distance. ④ Repeat steps ② and ③ until all vertices have been added to S.
- Worked example: Suppose there are 4 vertices A, B, C, D with source A and edge weights: A→B=2, A→C=5, B→C=1, B→D=3, C→D=1. a. Initialization: dist[A]=0, dist[B]=dist[C]=dist[D]=∞, S=∅ b. Add A to S and relax its neighbors: dist[B]=2, dist[C]=5 c. Add B to S and relax its neighbors: dist[C] updates to min(5, 2+1)=3, dist[D]=2+3=5 d. Add C to S and relax its neighbors: dist[D] updates to min(5, 3+1)=4 e. Add D to S; the algorithm ends. Final shortest paths: A→B=2, A→B→C=3, A→B→C→D=4

VI. Programming Question (10 points)

Reference implementation (singly linked list in C):

```c
#include <stdio.h>
#include <stdlib.h>

// Define the linked list node structure
typedef struct LNode {
    int data;           // data field
    struct LNode *next; // pointer field
} LNode, *LinkList;

// Create the list by head insertion
LinkList CreateListHead(int arr[], int n) {
    LinkList L = (LinkList)malloc(sizeof(LNode)); // head node
    L->next = NULL;
    for (int i = 0; i < n; i++) {
        LNode *p = (LNode *)malloc(sizeof(LNode));
        p->data = arr[i];
        p->next = L->next;
        L->next = p;
    }
    return L;
}

// Search by value; returns the node pointer, or NULL if not found
LNode* Search(LinkList L, int val) {
    LNode *p = L->next;
    while (p != NULL && p->data != val) {
        p = p->next;
    }
    return p;
}

// Insert value val at position pos; pos starts from 1
int Insert(LinkList L, int pos, int val) {
    LNode *p = L;
    int i = 0;
    // Find node pos-1
    while (p != NULL && i < pos - 1) {
        p = p->next;
        i++;
    }
    if (p == NULL) return 0; // invalid position
    LNode *s = (LNode *)malloc(sizeof(LNode));
    s->data = val;
    s->next = p->next;
    p->next = s;
    return 1;
}

// Delete the node at position pos
int Delete(LinkList L, int pos) {
    LNode *p = L;
    int i = 0;
    while (p->next != NULL && i < pos - 1) {
        p = p->next;
        i++;
    }
```

```
    if (p->next == NULL) return 0; // invalid position
    LNode *q = p->next;
    p->next = q->next;
    free(q);
    return 1;
}

// Print the list
void PrintList(LinkList L) {
    LNode *p = L->next;
    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }
    printf("\n");
}

// Functional tests
int main() {
    int arr[] = {1, 3, 5, 7, 9};
    LinkList L = CreateListHead(arr, 5);
    printf("Initial list: ");
    PrintList(L);

    Insert(L, 3, 4);
    printf("After inserting 4 at position 3: ");
    PrintList(L);

    Delete(L, 2);
    printf("After deleting position 2: ");
    PrintList(L);

    LNode *res = Search(L, 5);
    if (res) printf("Element 5 found\n");
    else printf("Element 5 not found\n");

    return 0;
}
```

II. Distribution of the Paper's Core Exam Points

Splitting the points by knowledge module shows the review priorities at a glance:

Knowledge module	Question types covered	Approx. points
Graphs (storage, traversal, shortest paths, minimum spanning tree, topological sort)	multiple-choice + fill-in + true-or-false + short-answer + comprehensive	24 points
Searching (hash tables, binary search trees)	multiple-choice + fill-in + true-or-false + short-answer	18 points
Trees and binary trees (properties, traversal, BST, AVL, Huffman)	multiple-choice + fill-in + true-or-false + short-answer	17 points

Knowledge module	Question types covered	Approx. points
Linear lists (arrays, linked lists)	multiple-choice + fill-in + true-or-false + programming	22 points
Sorting (algorithm principles, complexity analysis)	multiple-choice + fill-in + comprehensive	16 points
Stacks and queues	multiple-choice + fill-in + true-or-false + short-answer	13 points

Core conclusion: graphs, linear lists, searching, and trees are the four highest-value modules, with sorting and stacks/queues next; they are the core focus of the last-minute final-exam review.

III. Last-Minute Review Essentials and Study Directions

1. Review Priority Ranking (Highest to Lowest, Diminishing Returns)

- First priority (fastest score gains, must master): linked-list operations, hash tables, binary-tree traversal and properties, sorting-algorithm complexity, graph traversal and shortest paths

(Note: parts of this content may be AI-generated)

- Second priority (high-frequency concept questions): stack/queue properties, binary search trees, AVL / red-black tree concepts, minimum spanning tree, topological sort
- Third priority (can be strategically abandoned if time runs short): strings, generalized lists, files, implementing complex algorithm code

2. Core Last-Minute Techniques

1. Time complexity is the main thread running through the entire course

Almost every data structure and every algorithm gets tested on time / space complexity. Memorize the complexities of all the exam points above and the multiple-choice and fill-in-the-blank questions become direct points — the highest-value content for last-minute review.

2. Recite the concept-based exam points repeatedly

Most multiple-choice, fill-in-the-blank, and true-or-false questions are basic concepts (such as "methods for resolving hash collisions", "which data structure DFS uses", "which graphs topological sorting applies to"). These questions do not require deep understanding; reading them 3–5 times earns the points.

3. Memorize answer templates for short-answer / comprehensive questions

The exam points of final-exam short-answer and comprehensive questions are highly predictable (stack vs. queue, hash collisions, the principles of DFS/BFS, quicksort, Dijkstra). Memorize the answer points from the solutions above and apply them directly in the exam to earn 80% or more of the score.

4. For a guaranteed baseline on the programming question, drill the linked list

The programming question on this paper tests the linked list, which is also the first choice for data-structures final programming questions at the vast majority of institutions. During your last-minute review, be sure to

hand-write the creation, insertion, deletion, and search code of the singly linked list once; memorize the core template and the programming question becomes a guaranteed source of points.